

파이썬 기초~응용

『4과목 :』 데이터 분석과 인사이트 도출

2025.06.23-07.04(10일, 70시간)

Prepared by Daekyeong Kim

Ph.D.



1. 통계와 확률/ 통계 분석
2. 머신러닝 기반 데이터 분석-지도
3. 머신러닝 기반 데이터 분석-비지도
4. 데이터 마이닝
5. 기타 데이터 마이닝

6. 『4과목』 Self 점검

『 4과목 :』 데이터 분석과 인사이트 도출

- 통계와 확률 / 통계분석
 - 머신러닝 기반 데이터 분석-지도
 - 머신러닝 기반 데이터 분석-비지도
 - 기타 데이터 마이닝
-
- 『4과목』 Self 점검



학습목표

- 이 워크샵에서는 자율학습 모델, K-means 클러스터링(K-Means Clustering)기법에 대해 알 수 있습니다.

눈높이 체크

- K-means 클러스터링(K-Means Clustering) 을 알고 계시나요?



1. 자율학습 개념

자율학습?

- 자율학습 혹은 비지도 학습(Unsupervised Learning)머신러닝 기법은 데이터 세트에 목적변 수(혹은 반응변수)(Y)가 없이, 일련의 변수들 $X_1, X_2, X_3, \dots, X_p$ 만 주어진 경우에 시행하게 되는 머신러닝 기법들이다. 일련의 설명변수들과 연관된 목적변수(혹은 반응변수)가 없기 때문에, 무엇을 예측한다기보다는 주어진 데이터에서 특정한 패턴이나 알려지지 않은 지식을 발견하고자 하는 것이 목표라고 할 수 있다.
- 앞서 다룬 지도학습 머신러닝 기법들이 목적변수(혹은 반응변수)의 형태에 따라 분류 및 수치예측 수행이라는 명확한 목표가 있고, 데이터 세트 분할 통한 교차검증 등의 분석결과에 대한 명확한 평가 기준을 가지고 분석을 진행할 수 있는 것에 비해 자율학습 기법은 무엇을 발견하고자 하는 것인지 명확하지 않으며, 예측 대상을 '지도'할 수 없으므로, 컴퓨터 프로그램은 실제로 정답을 알 수 없다. 이로 인해 머신러닝 결과가 만족스러운 것인지 점검하기가 곤란하다는 문제가 있다.



1. 자율학습 개념

자율학습?

- 이러한 자율학습 머신러닝 기법은 분석의 목적 및 알고리즘에 따라, 유사한 개체나 사람들을 그룹 짓는 군집화(Clustering), 특정 대상들 간의 발생 관련성을 파악하는 연관성 분석(Association), 주어진 변수 세트를 효과적으로 설명 가능한 더 적은 수의 대표적인 변수들로 요약하는 차원축소(Dimension Reduction) 등의 기법이 있다.



2. 클러스터링(군집)

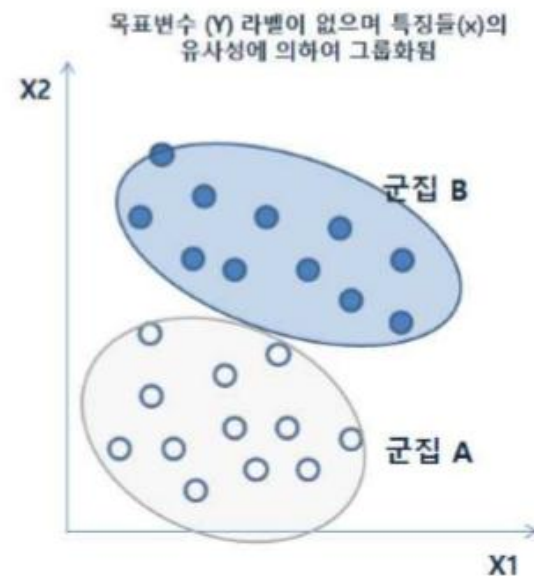
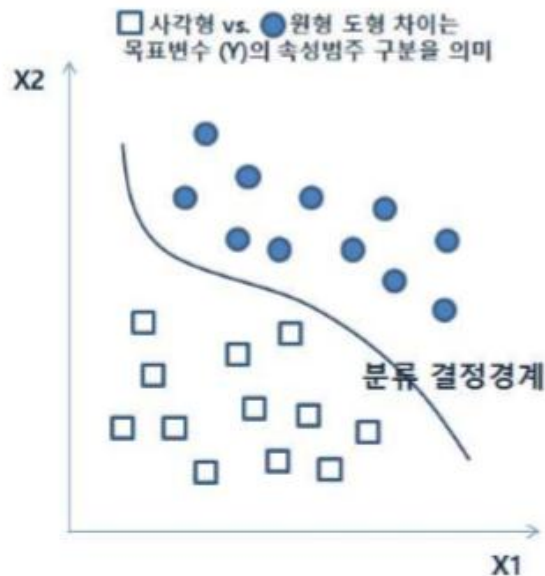
클러스터링(군집) 분석?

- 군집분석 혹은 클러스터링(Clustering) 분석은 대표적인 자율학습(혹은 비지도 학습)으로서, 일련의 관측값들을 적절한 기준으로 서로 유사한 관측값끼리 그룹으로 묶는(군집화) 기법을 말한다. 군집화는 사전에 그룹이 어떤 형태인지 모르는 상태에서 실행하게 된다. 분석가가 찾고 있는 것이 무엇인지 모르는 상태에서 실행하기 때문에 당연히 분석가가 컴퓨터 프로그램에게 무엇을 지도할 수 없다. 그런 의미에서 군집분석이 자율학습(비지도 학습)으로 불리고 있는 것이다. 또한, 동일한 이유로 군집분석은 무엇을 예측하기 위해 실행하기보다는 지식 발견 그 자체를 위한 목적으로 주로 활용된다. 그런 측면에서 보자면 데이터를 통해 학습하여 무엇을 예측하려는 머신러닝이라기 보다는 지식발견 그 자체 혹은 알려지지 않았던 통찰을 발견하려는 데이터 마이닝에 좀 더 가깝다고 할 수 있다.

2. 클러스터링(군집)

클러스터링(군집) 분석 원리

- 군집분석은 입력된 데이터가 어떤 형태로 그룹을 형성하는지가 분석의 핵심 목적이다. 따라서 군집분석에서는 입력된 데이터를 어떤 기준으로 그룹화 하는지가 첫 번째 질문이 되는데, 일반적으로는 각 데이터 간의 유사성을 기준으로 그룹화를 짓게 된다. 즉, 군집 내의 데이터는 서로 매우 유사하지만, 다른 군집과는 다르다는 원칙으로 그룹화를 진행하게 된다. 군집분석은 목적변수가 없는 상태에서 유사한 특성을 가진 개체끼리 그룹화를 한다. (A) 분류 목적 분석기법 (B) 군집 분석





2. 클러스터링(군집)

클러스터링(군집) 분석 주요 활용 분야

- 군집분석은 데이터들에 존재하는 유사성 혹은 비유사성에 근거해서 패턴화를 하는 특성으로 인해, 매우 다양한 영역에서 활용할 수 있다. 군집분석이 주로 활용되는 분야에 대한 대표적인 예시는 다음과 같다.
 1. 마케팅 등 분야에서의 고객 세분화(Segmentation)
 2. 질병 및 환자 특성에 따른 유사 그룹화
 3. 개체 유사성에 근거한 문서 분류
 4. 디지털 이미지 인식 통한 사물 및 안면 인식
 5. 금융 분야에서의 알려진 군집 이외의 사용 패턴 식별 (신용카드 사기, 보험료 과다 청구 등)
 6. 공학 분야에서의 이상치 탐지 (제조 과정에서의 불량제품 자동 탐지, 통화 음질 개선을 위한 노이즈 구별 등)
 7. 컴퓨터 네트워크에 비인가 된 침입 등의 비정상적 행위 탐지
- 상기 몇 가지 예시에서 볼 수 있는 바와 같이, 주로 유사한 사람들을 그룹화 및 세분화하여 비즈니스에 활용하거나, 유사한 개체들을 묶어 필요한 정보를 압축하여 활용하거나, 유사성 기반의 그룹화를 응용하여 역으로 이상치(Outlier) 등의 특이점 혹은 비정상 패턴을 찾는 분야 등에 활용되고 있다. 이 외에도 매우 다양한 영역에서 응용 및 활용되고 있는 중요한 분석 기법이라고 할 수 있다.

2. 클러스터링(군집)

클러스터링(군집) 분석의 주요 종류

구분	기법	주요 내용
비계층적 군집 (분할 기반 군집)	K-평균(K-Means) 클러스터링	주어진 군집 수 k 에 대해서 군집 내 거리 제곱 합을 최소화하는 형태로 데이터 내의 개체들을 서로 다른 군집으로 그룹화하는 기법
	K-Medoids 클러스터링 혹은 (PAM : Partitioning Around Method)	K-평균 클러스터링의 보완한 기법으로서, 모든 형태의 유사성(비유사성) 측도를 사용하며, 좌표평면상 임의의 점이 아닌 실제 데이터 세트 내의 값을 사용하여 클러스터 중심을 정하므로 노이즈나 이상치 처리에 강건한 군집화 기법
	DBSCAN (Density Based Spatial Clustering of Application with Noise)	K-평균 기법이 K 개의 평균과 각 데이터 점들 간의 거리를 계산하여 그룹화를 하는 반면, DBSCAN은 밀도 개념을 도입하여 일정한 밀도로 연결된 데이터 집합은 동일한 그룹으로 판정하여 노이즈 및 이상치 식별에 강한 군집화 기법
	자기 조직화 지도 (Self Organizing Map)	자율학습 목적의 머신러닝에 속하는 인공 신경망의 한 기법으로서 벡터 수량화 네트워크를 이용한 군집화 기법
	Fuzzy 군집	K-평균 기법이 하나의 데이터 개체는 하나의 군집에만 배타적으로 속하는 독점적 군집인데 반해, 퍼지군집은 하나의 데이터 개체가 여러 개의 군집에 중복해서 속할 수 있도록 하는 중복 군집화 기법
계층적 군집	병합적(Agglomerative) 혹은 상향식(Bottom-up) 군집화	모든 데이터 객체를 별개의 그룹으로 구성한 뒤, 단 하나의 그룹화가 될 때까지 각 그룹을 단계적으로 합쳐가는 계층적 군집기법
	분할식(Divisive) 혹은 하향식(Top-down) 군집화	모든 데이터 객체를 하나의 그룹으로 구성한 뒤, 각 데이터 점이 하나의 그룹으로 될 때까지 단계적으로 분할에 가는 계층적 군집기법
확률 기반 군집	가우스 혼합 모형	EM (Expectation Maximization) 알고리즘 혹은 MCMC (Markov Chain Monte Carlo) 등의 알고리즘을 사용하여 모수를 추정하는 확률 기반의 군집분석

2. 클러스터링(군집)

군집화 성능기준

- 군집화의 경우에는 분류문제와 달리 성능기준을 만들기 어렵다. 심지어는 원래 데이터가 어떻게 군집화되어 있었는지를 보여주는 정답(groundtruth)이 있는 경우도 마찬가지이다. 따라서 다양한 성능기준이 사용되고 있다. 다음의 군집화 성능기준의 예다.
 1. 조정 랜드지수(Adjusted Rand Index)
 2. 조정 상호정보량 (Adjusted Mutual Information)
 3. 실루엣계수 (Silhouette Coefficient)
- 일치행렬
 - 랜드지수를 구하려면 데이터가 원래 어떻게 군집화되어 있어야 하는지를 알려주는 정답(groundtruth)이 있어야 한다. N 개의 데이터 집합에서 i, j 두 개의 데이터를 선택하였을 때 그 두 데이터가 같은 군집에 속하면 1 다른 데이터에 속하면 0이라고 하자. 이 값을 $N \times N$ 행렬 T 로 나타내면 다음과 같다.

$$T_{ij} = \begin{cases} 1 & i \text{와 } j \text{가 같은 군집} \\ 0 & i \text{와 } j \text{가 다른 군집} \end{cases}$$

2. 클러스터링(군집)

군집화 성능기준

- 예를 들어 $\{0,1,2,3,4\}$ 라는 5개의 데이터 집합에서 $\{0,1,2\}$ 와 $\{3,4\}$ 가 각각 같은 군집라면 행렬 T 는 다음과 같다.

```
import numpy as np

groundtruth = np.array([
    [1, 1, 1, 0, 0],
    [1, 1, 1, 0, 0],
    [1, 1, 1, 0, 0],
    [0, 0, 0, 1, 1],
    [0, 0, 0, 1, 1],
])
groundtruth
```

- 이제 군집화 결과를 같은 방법으로 행렬 C 로 표시하자. 만약 군집화이 정확하다면 이 행렬은 정답을 이용해서 만든 행렬과 거의 같은 값을 가져야 한다. 만약 군집화 결과 $\{0,1\}$ 과 $\{2,3,4\}$ 가 같은 군집라면 행렬 C 는 다음과 같다.

```
clustering = np.array([
    [1, 1, 0, 0, 0],
    [1, 1, 0, 0, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 1],
])
clustering
```

2. 클러스터링(군집)

군집화 성능기준

- 이 두 행렬의 모든 원소에 대해 값이 같으면 1 다르면 0으로 계산한 행렬을 일치행렬 (incidence matrix)이라고 한다. 즉 데이터 집합에서 만들수 있는 모든 데이터 쌍에 대해 정답과 군집화 결과에서 동일한 값을 나타내면 1, 다르면 0이 된다.

$$R_{ij} = \begin{cases} 1 & \text{if } T_{ij} = C_{ij} \\ 0 & \text{if } T_{ij} \neq C_{ij} \end{cases}$$

- 즉, 원래 정답에서 1번 데이터와 2번 데이터가 같은(다른) 군집인데 군집화 결과에서도 같은(다른) 군집이라고 하면 $R_{12}=1$ 이다.
- 위 예제에서 일치행렬을 구하면 다음과 같다.

incidence = 1 * (groundtruth == clustering) # 1*는 True/False를 숫자 0/1로 바꾸기 위한 계산
incidence

>>

```
array([[1, 1, 0, 1, 1],  
       [1, 1, 0, 1, 1],  
       [0, 0, 1, 0, 0],  
       [1, 1, 0, 1, 1],  
       [1, 1, 0, 1, 1]])
```

2. 클러스터링(군집)

군집화 성능기준

- 이 일치 행렬은 두 데이터의 순서를 포함하므로 대칭행렬이다. 만약 데이터의 순서를 무시한다면 위 행렬에서 대각성분과 아래쪽 비대각 성분은 제외한 위쪽 비대각 성분만을 고려해야 한다. 위쪽 비대각 성분에서 1의 개수는 다음과 같아진다.
- $a=T$ 에서 같은 군집에 있고 C 에서도 같은 군집에 있는 데이터 쌍의 수
- $b=T$ 에서 다른 군집에 있고 C 에서도 다른 군집에 있는 데이터 쌍의 수
- 일치행렬 위쪽 비대각 성분에서 1의 개수 $= a+b$

```
np.fill_diagonal(incidence, 0) # 대각성분 제외  
a_plus_b = np.sum(incidence) / 2 # 대칭행렬이므로 절반만 센다.  
a_plus_b
```

>>

6.0

2. 클러스터링(군집)

군집화 성능기준

- 랜드지수
 - 랜드지수(Rand Index, RI)는 가능한 모든 데이터 쌍의 개수에 대해 정답인 데이터 쌍의 개수의 비율로 정의한다.

$$\text{Rand Index} = \frac{a + b}{N C_2}$$

- `shape[0]`, `shape[1]`를 이용하여 전체 행의 갯수와 열의 갯수를 반환

```
from scipy.special import comb
rand_index = a_plus_b / comb(incidence.shape[0], 2)
rand_index
```

```
>>
```

```
0.6
```

2. 클러스터링(군집)

K-평균 군집화(K-Means Clustering)

- K-평균 클러스터링은 주어진 군집 수 k 에 대하여 군집 내 거리 제곱 합의 합을 최소화하는 것을 목적으로 하며, 다음과 같은 목적함수 값이 최소화될 때까지 군집의 중심위치와 각 데이터가 소속될 군집을 반복해서 찾는다. 이 값을 관성(inertia)이라 한다.

$$J = \sum_{k=1}^K \sum_{i \in C_k} d(x_i, \mu_k)$$

- 이 식에서 K 는 군집의 갯수이고 C_k 는 k 번째 군집에 속하는 데이터의 집합, μ_k 는 k 번째 군집의 중심위치(centroid), d 는 x_i, μ_k 두 데이터 사이의 거리 혹은 비유사도(dissimilarity)로 정의한다. 만약 유클리드 거리를 사용한다면 다음과 같다.

$$d(x_i, \mu_k) = ||x_i - \mu_k||^2$$

-
- 위 식은 다음처럼 표현할 수도 있다.

$$J = \sum_{i=1}^N \min_{\mu_j \in C} (||x_i - \mu_j||^2)$$

2. 클러스터링(군집)

K-평균 군집화(K-Means Clustering)

- 세부 알고리즘은 다음과 같다.
 1. 임의의 중심위치 $\mu_k(k=1,\dots,K)$ 를 고른다. 보통 데이터 표본 중에서 K 개를 선택한다.
 2. 모든 데이터 $x_i(i=1,\dots,N)$ 에서 각각의 중심위치 μ_k 까지의 거리를 계산한다.
 3. 각 데이터에서 가장 가까운 중심위치를 선택하여 각 데이터가 속하는 군집을 정한다.
 4. 각 군집에 대해 중심위치 μ_k 를 다시 계산한다.
 5. 2 ~ 4를 반복한다.
- K-평균 군집화란 명칭은 각 군집의 중심위치를 구할 때 해당 군집에 속하는 데이터의 평균(mean)값을 사용하는데서 유래하였다. 만약 평균 대신 중앙값(median)을 사용하면 K-중앙값(K-Median) 군집화라 한다.

2. 클러스터링(군집)

K-평균 군집화(K-Means Clustering)

- scikit-learn의 cluster 서브패키지는 K-평균 군집화를 위한 KMeans 클래스를 제공한다. 다음과 같은 인수를 받을 수 있다.
 - ✓ n_clusters: 군집의 갯수
 - ✓ init: 초기화 방법. "random"이면 무작위, "k-means++"이면 K-평균++ 방법. 또는 각 데이터의 군집 라벨.
 - ✓ n_init: 초기 중심위치 시도 횟수. 디폴트는 10이고 10개의 무작위 중심위치 목록 중 가장 좋은 값을 선택한다.
 - ✓ max_iter: 최대 반복 횟수.
 - ✓ random_state: 시드값.
- 다음은 make_blobs 명령으로 만든 데이터를 2개로 K-평균 군집화하는 과정을 나타낸 것이다. 각각의 그림은 군집을 정하는 단계 3에서 멈춘 것이다. 마커(marker)의 모양은 소속된 군집을 나타내고 크기가 큰 마커가 해당 군집의 중심위치다. 각 단계에서 중심위치는 전단계의 군집의 평균으로 다시 계산되는 것을 확인할 수 있다.

2. 클러스터링(군집)

K-평균 군집화(K-Means Clustering)

```
import matplotlib as mpl
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

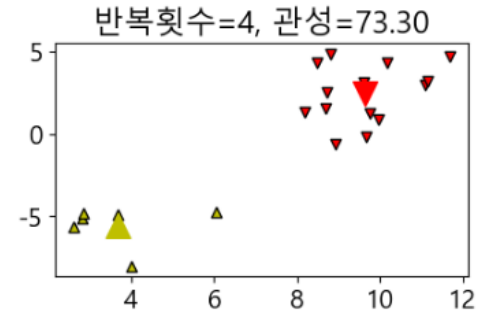
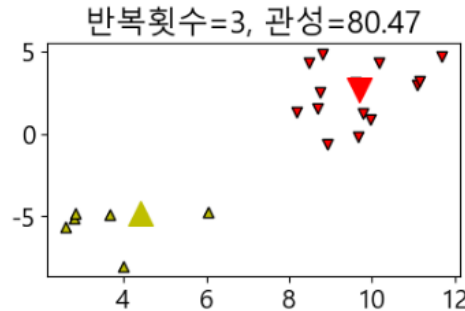
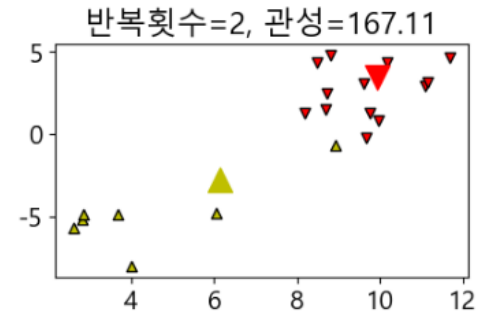
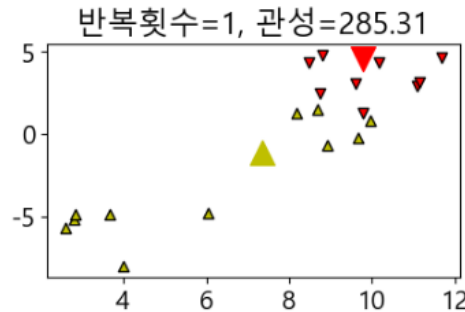
```
X, _ = make_blobs(n_samples=20, random_state=4)
```

```
def plot_KMeans(n):
    model = KMeans(n_clusters=2, init="random", n_init=1, max_iter=n, random_state=6).fit(X)
    c0, c1 = model.cluster_centers_
    plt.scatter(X[model.labels_ == 0, 0], X[model.labels_ == 0, 1], marker='v', facecolor='r',
edgecolors='k')
    plt.scatter(X[model.labels_ == 1, 0], X[model.labels_ == 1, 1], marker='^', facecolor='y',
edgecolors='k')
    plt.scatter(c0[0], c0[1], marker='v', c="r", s=200)
    plt.scatter(c1[0], c1[1], marker='^', c="y", s=200)
    plt.grid(False)
    plt.title("반복횟수={}, 관성={:5.2f}".format(n, -model.score(X)))
```

2. 클러스터링(군집)

K-평균 군집화(K-Means Clustering)

```
plt.figure(figsize=(8, 8))  
plt.subplot(321)  
plot_KMeans(1)  
plt.subplot(322)  
plot_KMeans(2)  
plt.subplot(323)  
plot_KMeans(3)  
plt.subplot(324)  
plot_KMeans(4)  
plt.tight_layout()  
plt.show()
```



- `plot_KMeans(n)`은 KMeans 모델을 훈련하고, 특정 반복 횟수에 따른 클러스터링 결과를 시각화

2. 클러스터링(군집)

K-평균++ 알고리즘

- K-평균++ 알고리즘은 초기 중심위치를 설정하기 위한 알고리즘이다. 다음과 같은 방법을 통해 되도록 멀리 떨어진 중심위치 집합을 찾아낸다.
 1. 중심위치를 저장할 집합 M 준비
 2. 일단 하나의 중심위치 μ_0 를 랜덤하게 선택하여 M 에 넣는다.
 3. M 에 속하지 않는 모든 표본 x_i 에 대해 거리 $d(M, x_i)$ 를 계산. $d(M, x_i)$ 는 M 안의 모든 샘플 μ_k 에 대해 $d(\mu_k, x_i)$ 를 계산하여 가장 작은 값 선택
 4. $d(M, x_i)$ 에 비례한 확률로 다음 중심위치 μ 를 선택.
 5. K 개의 중심위치를 선택할 때까지 반복
 6. K-평균 방법 사용
- 다음은 K-평균++ 방법을 사용하여 MNIST Digit 이미지 데이터를 군집화한 결과이다. 각 군집에서 10개씩의 데이터만 표시하였다.

2. 클러스터링(군집)

K-평균++ 알고리즘

```
from sklearn.datasets import load_digits

digits = load_digits()

model = KMeans(init="k-means++", n_clusters=10, random_state=0)
model.fit(digits.data)
y_pred = model.labels_

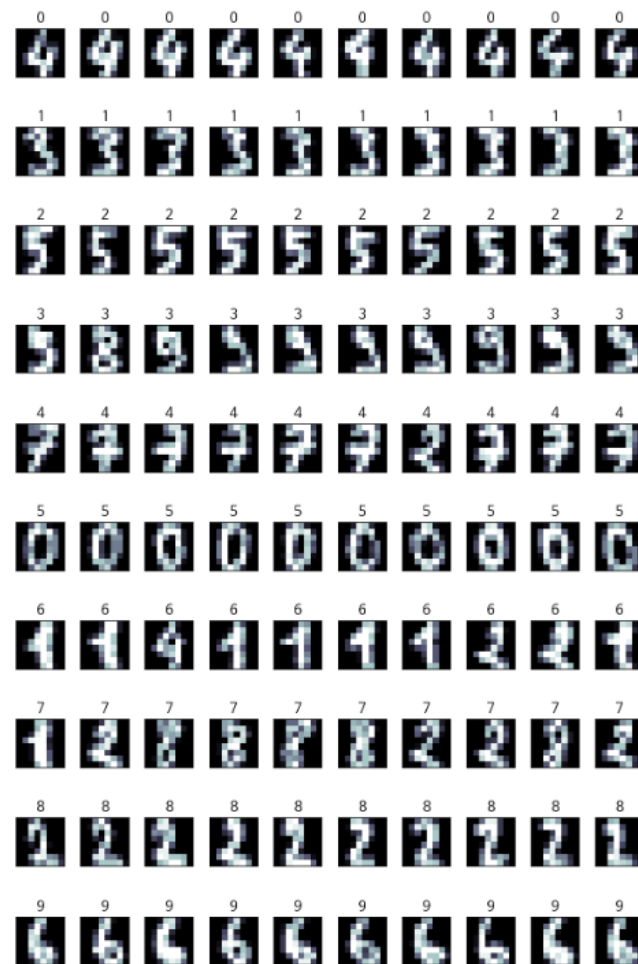
def show_digits(images, labels):
    f = plt.figure(figsize=(8, 2))
    i = 0
    while (i < 10 and i < images.shape[0]):
        ax = f.add_subplot(1, 10, i + 1)
        ax.imshow(images[i], cmap=plt.cm.bone)
        ax.grid(False)
        ax.set_title(labels[i])
        ax.xaxis.set_ticks([])
        ax.yaxis.set_ticks([])
        plt.tight_layout()
        i += 1

def show_cluster(images, y_pred, cluster_number):
    images = images[y_pred == cluster_number]
    y_pred = y_pred[y_pred == cluster_number]
    show_digits(images, y_pred)

for i in range(10):
    show_cluster(digits.images, y_pred, i)
```



- 각 클러스터마다 어떤 숫자들이 모여 있는지를 시각적으로 확인할 수 있습니다. 이 예제는 KMeans 알고리즘이 숫자 이미지 데이터를 어떻게 클러스터링하는지를 시각화하여 보여줍니다.



『 4과목 :』 데이터 분석과 인사이트 도출

- 통계와 확률 / 통계분석
- 머신러닝 기반 데이터 분석-지도
- 머신러닝 기반 데이터 분석-비지도
- 기타 데이터 마이닝 : Association Rule Analysis
- 『4과목』 Self 점검



학습목표

- 이 워크샵에서는 Association Rule Analysis 확인할 수 있다

눈높이 체크

- Association Rule Analysis 을 알고 계시나요?



1. 연관성 분석(장바구니 분석)

연관성 분석(장바구니 분석) 란?

- 연관성 분석 기법은 방대한 데이터 세트에서 객체나 아이템 간의 연관관계를 찾아내는 분석기법이다. Y값(종속변수, 목표변수)이 없는 상태에서 데이터 속에 숨겨져 있는 패턴, 규칙을 찾아내는 비지도학습(unsupervised learning)의 하나인 '연관규칙분석 (Association Rule Analysis)', 혹은 유통업계에서 사용하는 용어로 '장바구니분석(Market Basket Analysis)' 이라고도 한다.
- 상품 추천에 사용하는 분석기법
 1. 연관규칙분석, 장바구니분석 (Association Rule Analysis, Market Basket Analysis) : 고객의 대규모 거래데이터로부터 함께 구매가 발생하는 규칙(예: A à 동시에 B)을 도출하여, 고객이 특정 상품 구매 시 이와 연관성 높은 상품을 추천
 2. 순차분석 (Sequence Analysis) : 고객의 시간의 흐름에 따른 구매 패턴(A à 일정 시간 후 B)을 도출하여, 고객이 특정 상품 구매 시 일정 시간 후 적시에 상품 추천
 3. Collaborative Filtering : 모든 고객의 상품 구매 이력을 수치화하고, 추천 대상이 되는 고객A와 다른 고객B에 대해 상관계수를 비교해서, 서로 높은 상관이 인정되는 경우 고객B가 구입 완료한 상품 중에 고객A가 미구입한 상품을 고객A에게 추천



1. 연관성 분석(장바구니 분석)

연관성 분석(장바구니 분석) 란?

4. Contents-based recommendation : 고객이 과거에 구매했던 상품들의 속성과 유사한 다른 상품 아이템 중 미구매 상품을 추천 (↔ Collaborative Filtering은 유사 고객을 찾는 것과 비교됨)
 5. Who-Which modeling : 특정 상품(군)을 추천하는 모형을 개발 (예: 신형 G5 핸드폰 추천 스코어모형)하여 구매 가능성 높은(예: 스코어 High) 고객(군) 대상 상품 추천
- 넷플릭스에서 상품추천에 이용하는 알고리즘
 - 넷플릭스는 이용자들이 동영상에 매긴 별점과 위치정보, 기기정보, 플레이버튼 클릭 수, 평일과 주말에 따른 선호 프로그램, 소셜 미디어 내에서 언급된 횟수 등을 분석해 알고리즘을 개발했다.
 - 출처 : 넷플릭스의 빅데이터, 인문학적 상상력과의 접점, 조영신, KISDI 동향 Focus
 - 규칙(rule)이란 "if condition then result" (if A --> B) 의 형식으로 표현을 합니다.
 - 연관규칙(association rule)은 특정 사건이 발생하였을 때 함께 (빈번하게) 발생하는 또 다른 사건의 규칙을 말합니다.



1. 연관성 분석(장바구니 분석)

연관성 분석(장바구니 분석) 란?

- 대량의 트랜잭션 데이터에서의 규칙성이나 패턴화 도출의 목적이 주어진 경우, 이의 문제 해결을 위해 다양한 연관 규칙 알고리즘을 비교하고 적용할 수 있다.
- 연관성 분석 기법은 방대한 데이터 세트에서 객체나 아이템 간의 연관관계를 찾아내는 분석기법이다. 연관관계는 빈발 아이템이나 연관규칙의 형태로 표현되는데 빈발 아이템은 $\{X, Y\}$ 처럼 표현하고, 연관 규칙은 $\{X\} \rightarrow \{Y\}$ 처럼 특정 아이템 'X'가 발생하면, 'Y'가 함께 발생한다는 형태로 표현한다. 예를 들어 $\{\text{순대, 족발}\} \rightarrow \{\text{보쌈}\}$ 처럼 순대와 족발을 구매하면 보쌈도 함께 구매한다는 규칙을 찾아내는 것이다. 여기서 $\{\text{순대, 족발}\}$ 은 해당 규칙에서의 '조건'(antecedent)에 해당하고, $\{\text{보쌈}\}$ 은 '결과'(consequence)에 해당하게 된다. 이러한 연관성 분석은 사전에 목표변수(Y)가 주어지지 않으며, 각 트랜잭션(혹은 거래데이터)에서 객체나 아이템 간의 패턴을 찾아내는 기법이므로 자율학습(비지도 학습) 기법에 속하게 된다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 척도

- 연관성 분석에서 가장 핵심적인 개념은 각 아이템 간의 연관성을 파악하는 주요 3개 척도인 지지도(Support), 신뢰도(Confidence), 향상도(Lift)이다.
- 지지도(Support)
 - 지지도는 전체 데이터 세트에서 해당 아이템 집합이 포함된 비율을 말하며 아래의 $S(X)$ 혹은 $S(X,Y)$ 와 같이 표현된다.

$$S(X) = \frac{\text{count}(X)}{N} \quad (4.11)$$

$$S(X, Y) = \frac{\text{count}(X, Y)}{N} = P(X \cap Y)$$

- 즉, 지지도는 빈도적 관점에서 확률을 정의할 때, 전체 데이터 세트 중 아이템 집합 $\{X,Y\}$ 가 발생할 확률과 같다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 척도

- 신뢰도(Confidence)
- 신뢰도는 연관규칙 $\{X\} \rightarrow \{Y\}$ 에서 '조건' X 를 포함한 아이템 세트 중에서 X, Y 둘 다 포함된 아이템 세트가 발생한 비율을 말하는데, 규칙의 왼쪽에 있는 '조건 X '가 발생했다는 조건하에 규칙의 오른쪽에 있는 '결과 Y '가 발생할 확률을 의미한다. 신뢰도는 특정 연관 규칙의 예측력이나 정확도에 대한 측정이다. 사실 이 신뢰도는 조건부 확률 $P(Y|X)$ 와 동일한 의미이다.

$$\begin{aligned} Conf(X \Rightarrow Y) &= \frac{S(X, Y)}{S(X)} = \frac{\frac{count(X, Y)}{N}}{\frac{count(X)}{N}} = \frac{count(X, Y)}{count(X)} \quad (4.12) \\ &= \frac{P(X \cap Y)}{P(X)} = P(Y|X) \end{aligned}$$

- 신뢰도 $(X \rightarrow Y)$ 와 신뢰도 $(Y \rightarrow X)$ 는 서로 같지 않다. 즉, 두 신뢰도 모두 X, Y 가 모두 포함된 지지도(X, Y)가 분자에 포함되어 있지만 신뢰도 $(X \rightarrow Y)$ 는 지지도(X)가 분모에 들어가게 되고, 신뢰도 $(Y \rightarrow X)$ 는 지지도(Y)가 분모에 들어가게 되는 점이 다르다. 결국 이는 조건부 확률 $P(Y|X)$ 와 $P(X|Y)$ 가 서로 다른 결과를 가져오는 것과 동일한 의미라고 할 수 있다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 척도

- 향상도(Lift)
- 지지도와 신뢰도는 연관규칙 생성과 탐색에 있어서 매우 중요한 핵심개념임에는 틀림없지만, 지지도(X, Y)와 신뢰도($X \rightarrow Y$)가 높았다는 것만으로 유의미한 규칙이라고 결론 내리기는 어렵다. 그 이유는 만일 전체 데이터 세트에서 원래부터 아이템 세트 $\{Y\}$ 가 포함된 경우의 수가 많았다면, 아이템 세트 $\{Y\}$ 와 함께 아이템 세트 $\{X\}$ 가 포함되어 있을 가능성도 그만큼 커지고, 지지도(X, Y)와 신뢰도($X \rightarrow Y$)는 높게 나타날 수밖에 없기 때문이다.
- 즉, 연관규칙 $\{X\} \rightarrow \{Y\}$ 가 탐색 되었을 때 정말로 조건 $\{X\}$ 가 발생했을 때 결과 $\{Y\}$ 가 함께 나타나는 경우의 수가 많아서 그런 것인지, 아니면 원래부터 $\{Y\}$ 가 많이 포함되어 있어 연관규칙 $\{X\} \rightarrow \{Y\}$ 가 탐색 된 것인지에 대한 보다 명확한 측정 지표가 필요하게 된다. 이럴 때 이를 측정하는 지표가 바로 향상도(Lift)이다.
- 향상도(Lift)가 의미하는 바는 조건 $\{X\}$ 가 주어지지 않았을 때의 결과 $\{Y\}$ 가 발생할 확률 대비, 조건 $\{X\}$ 가 주어졌을 때의 결과 $\{Y\}$ 의 발생 확률의 증가 비율을 의미한다. 즉, 아이템 집합 $\{Y\}$ 가 원래 발생된 경우의 수보다 연관규칙 $\{X\} \rightarrow \{Y\}$ 가 탐색 되었을 때 조건에 해당하는 아이템 집합 $\{X\}$ 가 주어졌다는 "정보"가 결과 아이템 집합 $\{Y\}$ 가 발생하게 되는 경우의 수를 예상하는 데 얼마나 유용하느냐를 나타낸다고 할 수 있다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 척도

- 향상도(Lift)
- 향상도(Lift)는 다음과 같이 측정된다.

$$Lift(X \Rightarrow Y) = \frac{Conf(X \Rightarrow Y)}{S(Y)} \quad (4.13)$$

$$= \frac{\frac{S(X, Y)}{S(X)}}{S(Y)} = \frac{S(X, Y)}{S(X)S(Y)} = \frac{\frac{count(X, Y)}{N}}{\frac{count(X)}{N} \frac{count(Y)}{N}}$$

$$= \frac{P(Y|X)}{P(Y)} = \frac{\frac{P(X \cap Y)}{P(X)}}{P(Y)} = \frac{P(X \cap Y)}{P(X)P(Y)}$$

- 따라서 향상도가 1을 넘어가면 조건과 결과 아이템 집합 간에 서로 양의 상관관계가 있으며, 1보다 작으면 서로 음의 상관관계 만일 향상도가 1로 나타나면 조건과 결과 아이템 집합은 서로 독립적인 관계라고 할 수 있다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 척도

• 예

지지도(Support), 신뢰도(Confidence), 향상도(Lift) 예시

Customer ID	Transaction ID	Items
1131	1번	계란, 우유
2094	2번	계란, 기저귀, 맥주, 사과
4122	3번	우유, 기저귀, 맥주, 콜라
4811	4번	계란, 우유, 맥주, 기저귀
8091	5번	계란, 우유, 맥주, 콜라

↓
N = 5 (전체 transaction 개 수)

$$s(Y) = n(Y) / N \\ = n(2번, 3번, 4번) / N = 3/5 = 0.6$$

연관규칙 {계란, 맥주} → {기저귀} 에 대해
X Y

▪ 지지도(Support)

$$s(X \rightarrow Y) = n(X \cup Y) / N \\ = n(2번, 4번) / N \\ = 2/5 = 0.4$$

▪ 신뢰도(Confidence)

$$c(X \rightarrow Y) = n(X \cup Y) / n(X) \\ = n(2번, 4번) / n(2번, 4번, 5번) \\ = 2/3 = 0.667$$

▪ 향상도(Lift)

$$Lift(X \rightarrow Y) = c(X \rightarrow Y) / s(Y) \\ = 0.667 / 0.6 = 1.111$$



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 축도

- 연관성 분석 기법의 주요 활용 분야
 - 연관성 분석은 소매업 혹은 쇼핑몰 등의 거래데이터에서 물품 간에 연관관계를 파악하는 '장바구니 분석'에 가장 많이 사용되어 왔기 때문에 마케팅 영역에서는 연관성 분석 = 장바구니 분석으로 거의 동일한 용어로 사용되기도 한다. 그러나 연관성 분석이 비단 장바구니 분석에만 활용되는 것은 아니며, 객체(혹은 문서) 및 아이템 간의 연관 관계성 혹은 빈출 패턴을 파악할 필요가 있는 다양한 영역에서 활용되고 있다.
 1. 쇼핑/유통 분야에서 구매된 물품 간의 장바구니 분석
 2. 금융상품이나 서비스 간의 가입 패턴의 연관성 분석
 3. 인터넷/모바일 서비스 상품 추천 시스템의 구매 항목 간 연관성 알고리즘으로 사용
 4. 웹 페이지 간의 링크 관계성 분석 (웹 사용자의 행동 추적 및 패턴 예측)
 5. 가맹점 간의 방문 연관성 및 순서 패턴 도출 통한 쿠폰 마케팅 등 활용
 6. 생물 정보학 분야에서의 단백질과 유전자 패턴 분석
 7. 신용카드 사기 및 보험청구 사기 등 부정거래 패턴 발견
 - 즉, 방대한 데이터베이스나 인터넷 로그 데이터 등에서 아이템, 객체들 간의 발생 연관성을 파악하거나 발생 순서 간 패턴을 파악하려는 거의 모든 분야에서 활용 가능한 현실 세계의 실무 활용성이 높은 기법이라고 할 수 있다.



1. 연관성 분석(장바구니 분석)

연관성 분석 주요 측도

- 연관성 분석 알고리즘
- 연관성 분석의 대표적인 알고리즘에는 아프리오리(Apriori) 알고리즘과 이를 개선한 빈출 패턴 성장(FP-Growth) 알고리즘이 있다.

알고리즘	알고리즘 설명
아프리오리(Apriori) 알고리즘	빈출 아이템 집합을 효과적으로 계산하기 위하여 검토해야 할 대상 집합 Pool을 효과적으로 줄여주는 기법. 즉, 특정 집합 $\{1,2\}$ 가 빈발하지 않는다면, 이들을 포 함한 $\{0,1,2\}$, $\{1,2,3\}$, $\{1,2,4\}$, $\{0,1,2,3,4\}$ 등도 빈발하지 않은 것이 되어, 지지도 계 산 검토 대상에서 제외할 수 있다. 이런 식으로 검토대상의 아 이템 데이터 집합 을 효과적으로 줄여서 연관규칙을 계산해내는 기법. 데이터가 커질 경우 계산량이나 속도 면에서 비효율적이라는 단점이 있음
빈출패턴 성장 (FP-Growth)	아프리오리 알고리즘의 빈발 아이템을 찾는 방법을 개선한 알고리즘으로서 데 이 터베이스를 모든 중요한 정보를 가진 FP-Tree라는 구조로 압축한 뒤, 높은 빈도 의 세트에 관련한 조건부 데이터 세트로 분할하여 규칙을 만들어 낸다. 전 체 데 이터베이스를 검색하는 작업이 아프리오리 알고리즘보다 현저하게 줄어 들어서 높은 성능과 처리속도를 보이는 알고리즘임. 반면, 아프리오리 알고리즘 보다 구 현이 어렵다는 단점이 있음.
DHP algorithm	Transaction 개수(N)을 줄이는 전략

2. 연관성 분석 알고리즘

연관성 분석 주요 척도

- 연관성 분석 알고리즘

Association Rule : strategy and algorithm



- 1 모든 가능한 항목집합 개수(M)를 줄이는 방식 ➡ Apriori algorithm
- 2 Transaction 개수(N)를 줄이는 방식 ➡ DHP Algorithm
- 3 비교하는 수(W)를 줄이는 방식 ➡ FP-growth Algorithm

2. 연관성 분석 알고리즘

Apriori algorithm

- 최소지지도 이상을 갖는 항목집합을 빈발항목집합(frequent item set)이라고 합니다. 모든 항목집합에 대한 지지도를 계산하는 대신에 최소 지지도 이상의 빈발항목집합만을 찾아내서 연관규칙을 계산하는 것이 Apriori algorithm의 주요 내용입니다.
- 빈발항목집합 추출의 Apriori Principle
 1. 한 항목집합이 빈발(frequent)하다면 이 항목집합의 모든 부분집합은 역시 빈발항목집합이다.(frequent item sets -> next step)
 2. 한 항목집합이 비빈발(infrequent)하다면 이 항목집합을 포함하는 모든 집합은 비빈발항목집합이다. (superset -> pruning)
- Apriori Pruning Principle

If there is any itemset which is infrequent, its superset should not be generated/tested!

(Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)

2. 연관성 분석 알고리즘

Apriori algorithm

- {A, B, C, D, E}의 5개 원소 항목을 가지는 4건의 transaction에서 minimum support 2건 (=2건/총4건=0.5) 기준으로 pruning 하는 예

▪ Pruning criteria (minimum support) : $\text{Sup}_{\min} = 2$



2. 연관성 분석 알고리즘

Apriori algorithm

- 빈발항목집합 추출 Pseudo Aprior algorithm

```
1)  $L_1 = \{\text{large 1-itemsets}\};$ 
2) for ( $k = 2; L_{k-1} \neq 0; k++$ ) do begin
3)    $C_k = \text{apriori-gen}(L_{k-1});$  // New candidates
4)   forall transactions  $t \in T$  do begin
5)      $C_t = \text{subset}(C_k, t);$  // Candidates contained in  $t$ 
6)     forall candidates  $c \in C_t$  do
7)        $c.\text{count}++;$ 
8)   end
9)    $L_k = \{c \in C_k \mid c.\text{count} \geq \text{min sup}\}$ 
10) end
11) Answer =  $\bigcup_k L_k;$ 
```

k-itemset	[Notation] An itemset having k items
L_k	Set of large k-itemsets (those with minimum support) Each member of this set has two fields: i) itemset and ii) support count.
C_k	Set of candidate k-itemsets (potentially large itemsets) Each member of this set has two fields: i) itemset and ii) support count
$\overline{C}_k$	Set of candidate k-itemsets when the TIDs of the generating transactions are kept associated with the candidates

2. 연관성 분석 알고리즘

Apriori 예

1. mlxtend 패키지 설치

```
pip install mlxtend
```

2. 패키지 로드

```
# 패키지 로드
```

```
import pandas as pd  
from mlxtend.preprocessing import TransactionEncoder  
from mlxtend.frequent_patterns import apriori
```




2. 연관성 분석 알고리즘

Apriori 예

3. 데이터셋 생성

데이터셋 생성

```
dataset=[['사과','치즈','생수'],  
['생수','호두','치즈','고등어'],  
['수박','사과','생수'],  
['생수','호두','치즈','옥수수']]  
dataset
```

>>

```
[['사과', '치즈', '생수'],  
 ['생수', '호두', '치즈', '고등어'],  
 ['수박', '사과', '생수'],  
 ['생수', '호두', '치즈', '옥수수']]
```

2. 연관성 분석 알고리즘

Apriori 예

4. 가나다 순으로 Column값을 생성해서(고등어,사과,생수,수박,옥수수,치즈,호두) 첫번째 (사과,치즈.생수) 데이터에 고등어가 표시되어있으면 True값으로 없으면 False값으로 표시한다. 다음 사과도 첫번째 데이터에 사과가 있으면 1값으로 없으면 0값으로 표시한다. 이러한 것을 반복하면 4X7 행렬이 생성된다

```
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_) #위에서 나온걸 보기 좋게 데이터프레임으로 변경
df
>>
```

	고등어	사과	생수	수박	옥수수	치즈	호두
0	False	True	True	False	False	True	False
1	True	False	True	False	False	True	True
2	False	True	True	True	False	False	False
3	False	False	True	False	True	True	True

2. 연관성 분석 알고리즘

Apriori 예

5. 지지도를 0.5로 놓고 apriori를 돌려보면 아래와 같이 결과가 출력된다.
사과를 살 확률 0.5, 치즈 등을 함께 살 확률 0.75

```
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
frequent_itemsets
```

>>

	support	itemsets
0	0.50	(사과)
1	1.00	(생수)
2	0.75	(치즈)
3	0.50	(호두)
4	0.50	(생수, 사과)
5	0.75	(치즈, 생수)
6	0.50	(호두, 생수)
7	0.50	(치즈, 호두)
8	0.50	(치즈, 호두, 생수)

2. 연관성 분석 알고리즘

Apriori 예

6. 신뢰도가 0.3인 항목만 보기

```
from mixtend.frequent_patterns import association_rules
association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3
```

```
>>
```

	antecedents leverage	consequents conviction	antecedent support zhangs_metric		consequent support		support	confidence	lift
0	(생수) 0.0	(사과)	1.00	0.50	0.50	0.500000	1.000000	0.000	1.0
1	(사과) 0.0	(생수)	0.50	1.00	0.50	1.000000	1.000000	0.000	inf
2	(치즈) 0.0	(생수)	0.75	1.00	0.75	1.000000	1.000000	0.000	inf
3	(생수) 0.0	(치즈)	1.00	0.75	0.75	0.750000	1.000000	0.000	1.0
4	(호두) 0.0	(생수)	0.50	1.00	0.50	1.000000	1.000000	0.000	inf
5	(생수) 0.0	(호두)	1.00	0.50	0.50	0.500000	1.000000	0.000	1.0
6	(치즈) 1.0	(호두)	0.75	0.50	0.50	0.666667	1.333333	0.125	1.5
7	(호두) 0.5	(치즈)	0.50	0.75	0.50	1.000000	1.333333	0.125	inf
8	(치즈, 호두) 0.0	(생수)	0.50	1.00	0.50	1.000000	1.000000	0.000	inf
9	(치즈, 생수) 1.0	(호두)	0.75	0.50	0.50	0.666667	1.333333	0.125	1.5



2. 연관성 분석 알고리즘

Apriori 예

결론

신뢰도는 위에 표처럼 해석 할 수 있는데 예를 들면 10번째 열을 보면 (생수,호두)를 구매한 고객이 (치즈)를 구매할 확률이 높다고 판단 할 수 있다

항상도(lift)	의미
1 보다 작은 경우	우연적 기회보다 높은 확률
1 인 경우	독립
1 보다 큰 경우	우연적 기회보다 낮은 확률

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

- FP-growth 알고리즘을 적용시키기 위해서 가장 먼저 해야 할 일은 FP-tree를 만드는 것이다. 이 FP-tree를 만듦으로써 우리는 기존에 데이터셋을 모두 찾아보면서 찾아야했던 frequent itemset들을 FP-tree 하나에서만 찾을 수 있게 된다.
- 우선, 우리에게 다음과 같은 데이터셋이 있다고 하자.

$$D = \left[\left\{ \text{초장} \right\}^9, \left\{ \text{과메기} \right\}^{40}, \left\{ \text{과메기}, \text{김} \right\}^{50}, \left\{ \text{과메기}, \text{김}, \text{초장} \right\}^1 \right]$$

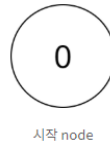
- 예시 데이터셋. 초장만 산 사람이 9명, 과메기만 산 사람이 40명, 과메기와 김을 함께 산 사람이 50명, 과메기, 김, 초장을 모두 함께 산 사람이 1명이라는 뜻이다.
1. 모든 각각의 item들에 대해 전체 데이터셋에서 나온 빈도를 계산한다.
 - ✓ 데이터셋의 모든 item들에 대해 빈도를 계산한다. 우리의 예시에서는 초장이 $9+1=10$ 으로 10번, 과메기가 $40+50+1=91$ 번, 김이 $50+1=51$ 번 나타났다. 이를 빈도가 높은 순서대로 표현하면 과메기:91, 김:51, 초장:10이다.

2. 연관성 분석 알고리즘

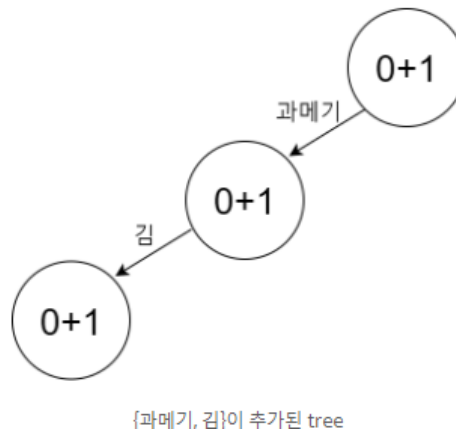
FP-Growth* Algorithm (Frequent-Pattern Growth)

2. itemset 하나씩 tree에 더함으로써 tree를 만들어준다.

- ✓ 이제 tree를 만들 차례이다. 우선, 맨 처음에 0 node를 하나 만든다.



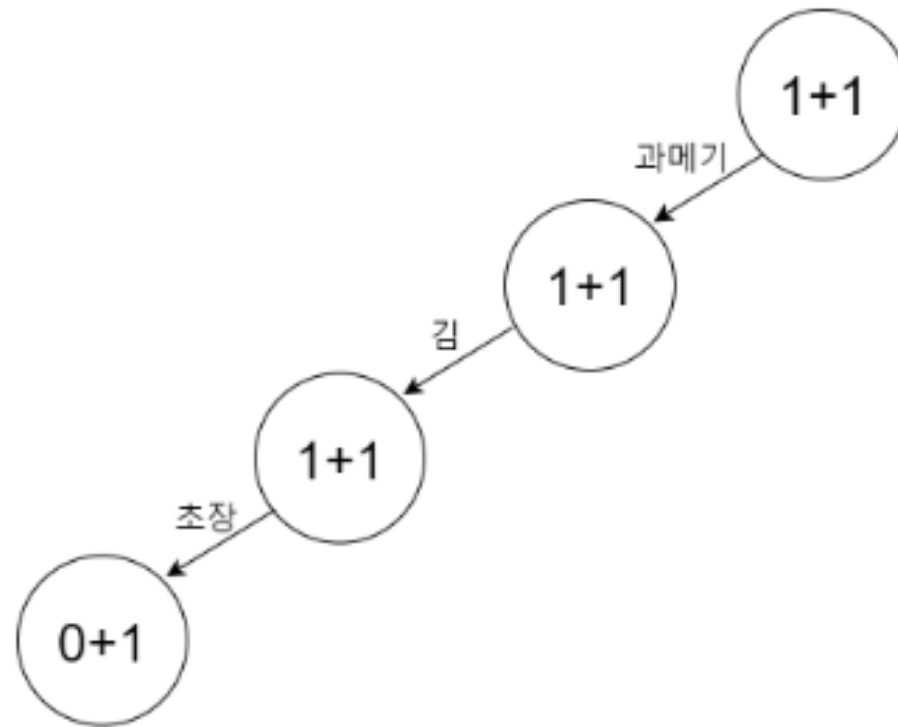
- ✓ 그리고 각 instance를 하나씩 더해주면서 tree를 만들어간다. 이 때, 빈도가 높은 item이 더 0에 가까운 node가 된다. 예를 들어, 우선 {과메기, 김} itemset으로 tree를 만든다고 하자. 그러면 다음과 같은 형태가 된다.



2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

- ✓ 빈도가 더 높은 과메기가 더 가까운 곳에 위치하는 tree가 만들어졌다. 이 단계에서 우리가 {과메기, 김, 초장} itemset을 추가한다고 하자. 그러면 다음과 같은 형태가 될 것이다.

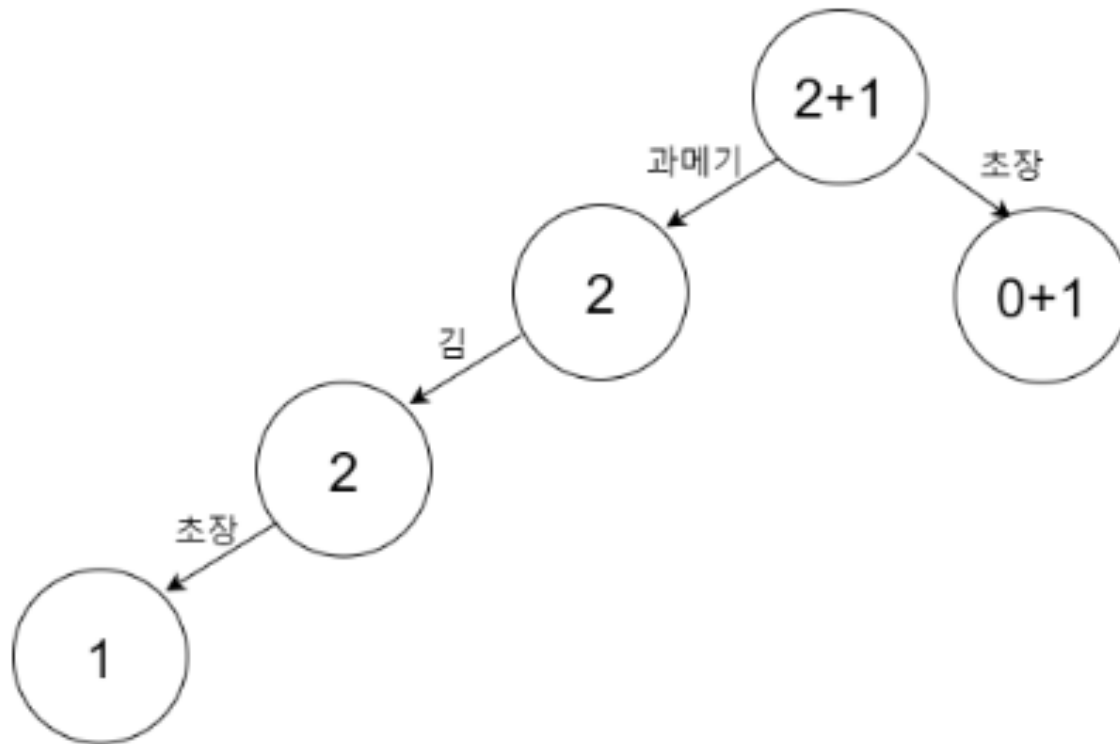


{과메기, 김, 초장}이 추가된 tree

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

- ✓ 여기서 {초장} itemset을 추가한다고 하자. 그러면 다음과 같은 형태가 될 것이다.

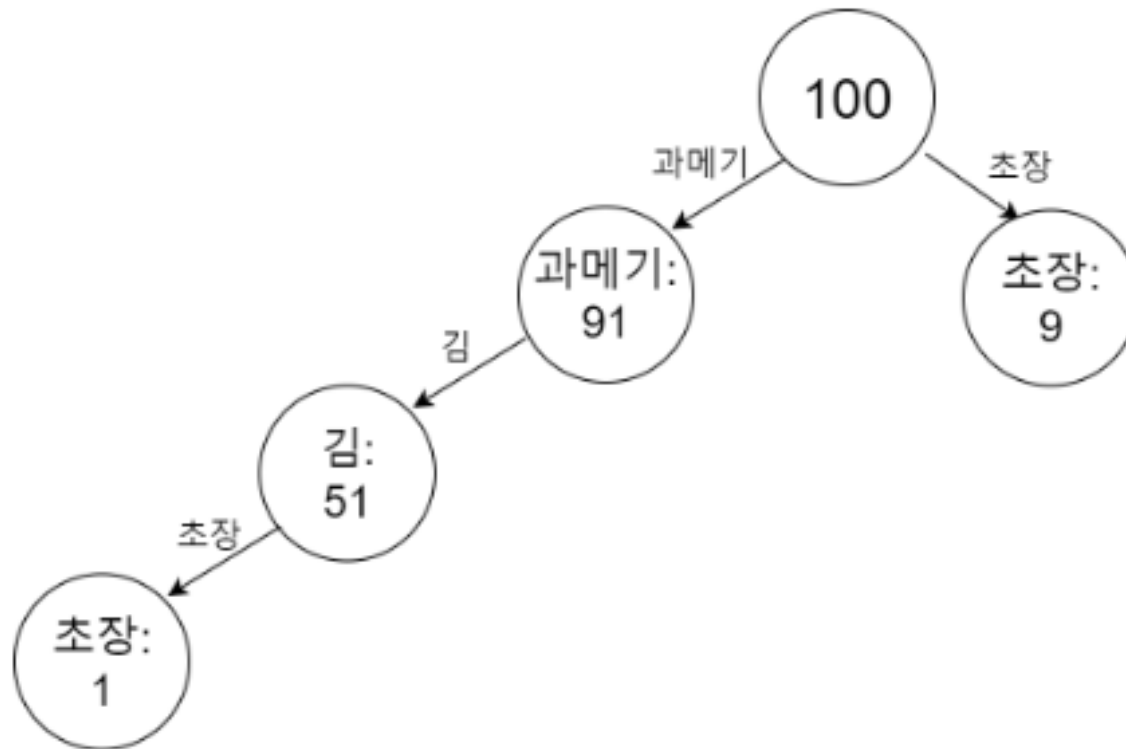


{초장}이 추가된 tree

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

- ✓ 이런 식으로 dataset 안의 모든 itemset에 대해 tree를 만들 수 있다. 그러면 최종적으로 다음과 같은 FP-tree를 얻을 수 있다.



최종 FP-tree



2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

- ✓ 이렇게 dataset 안의 모든 itemset의 정보를 반영하는 FP-tree를 만들어낼 수 있다. 이 FP-tree를 이용하면 아주 쉽게 support 값을 구할 수 있다. 예를 들어, 우리의 tree에서 {김, 과메기}의 support를 구하고 싶다고 하자. 김은 무조건 과메기와 함께 나타나기 때문에 김의 frequency만 고려하면 되고, 이는 51이므로 {김, 과메기}의 support는 51이 된다. 이러한 방법으로 우리의 dataset의 support를 모두 구하면 다음과 같다.

item sets	support
{과메기}	91
{김}, {김, 과메기}	50
{초장}	10
{과메기, 김, 초장}	1

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

```
pip install mlxtend --upgrade
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
```

```
Requirement already satisfied: mlxtend in /usr/local/lib/python3.7/dist-packages (0.14.0)
```

```
Collecting mlxtend
```

```
  Downloading mlxtend-0.20.0-py2.py3-none-any.whl (1.3 MB)
```

```
    | 1.3 MB 4.8 MB/s
```

```
Requirement already satisfied: setuptools in /usr/local/lib/python3.7/dist-packages (from mlxtend) (57.4.0)
```

```
Requirement already satisfied: scikit-learn>=1.0.2 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (1.0.2)
```

```
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (3.2.2)
```

```
Requirement already satisfied: numpy>=1.16.2 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (1.21.6)
```

```
Requirement already satisfied: scipy>=1.2.1 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (1.4.1)
```

```
Requirement already satisfied: pandas>=0.24.2 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (1.3.5)
```

```
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.7/dist-packages (from mlxtend) (1.1.0)
```

```
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.7/dist-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)
```

```
Requirement already satisfied: python-dateutil>=2.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib>=3.0.0->mlxtend) (2.8.2)
```

```
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib>=3.0.0->mlxtend) (3.0.9)
```

```
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.7/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.4.2)
```

```
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.7/dist-packages (from kiwisolver>=1.0.1->matplotlib>=3.0.0->mlxtend) (4.2.0)
```

```
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/dist-packages (from pandas>=0.24.2->mlxtend) (2022.1)
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/dist-packages (from python-dateutil>=2.1->matplotlib>=3.0.0->mlxtend) (1.15.0)
```

```
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn>=1.0.2->mlxtend) (3.1.0)
```

```
Installing collected packages: mlxtend
```

```
  Attempting uninstall: mlxtend
```

```
    Found existing installation: mlxtend 0.14.0
```

```
    Uninstalling mlxtend-0.14.0:
```

```
      Successfully uninstalled mlxtend-0.14.0
```

```
Successfully installed mlxtend-0.20.0
```

```
WARNING: The following packages were previously imported in this runtime:
```

```
  [mlxtend]
```

```
You must restart the runtime in order to use newly installed versions.
```

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

```
pip install mlxtend --upgrade
```

```
from sklearn.preprocessing import MultiLabelBinarizer
```

```
data = [  
    ['휴지', '물티슈', '샴푸'],  
    ['수세미', '물티슈', '비누'],  
    ['휴지', '수세미', '물티슈', '비누'],  
    ['수세미', '비누']  
]
```

```
mlb = MultiLabelBinarizer()  
encoded_data = mlb.fit_transform(data)  
df_encoded = pd.DataFrame(encoded_data, columns=mlb.classes_)
```

```
print(df_encoded)
```

```
>>
```

```
물티슈 비누 샴푸 수세미 휴지  
0  1  0  1  0  1  
1  1  1  0  1  0  
2  1  1  0  1  1  
3  0  1  0  1  0
```

2. 연관성 분석 알고리즘

FP-Growth* Algorithm (Frequent-Pattern Growth)

```
from mixtend.frequent_patterns import fpgrowth
fpgrowth(df_encoded, min_support=0.5, use_colnames=True)
>>
```

	support	itemsets
0	0.75	(물티슈)
1	0.50	(휴지)
2	0.75	(수세미)
3	0.75	(비누)
4	0.50	(비누, 물티슈)
5	0.50	(물티슈, 수세미)
6	0.50	(비누, 물티슈, 수세미)
7	0.50	(휴지, 물티슈)
8	0.75	(비누, 수세미)

THANK YOU.

